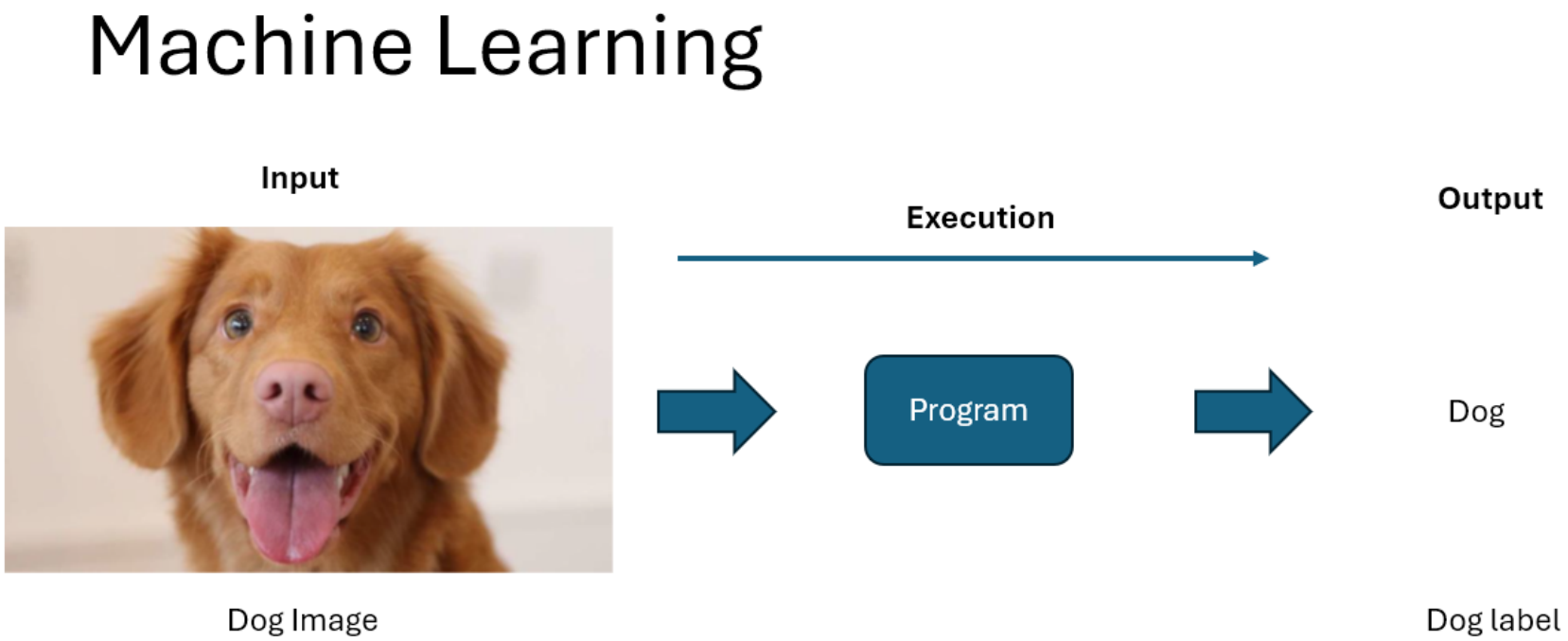
CS515: Introduction to Deep Learning

Emre Ozfatura[1]

[1] Intelligent Systems Lab,
Sabanci University, Turkey

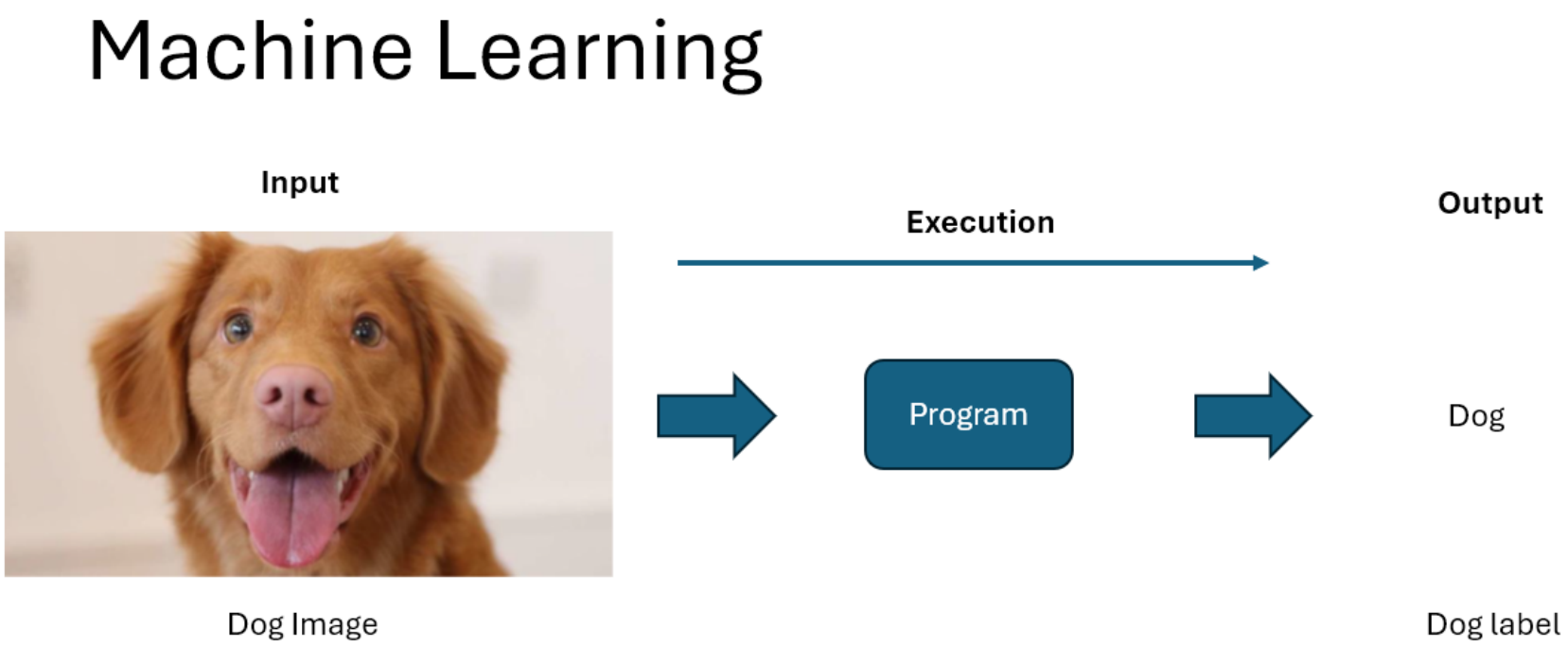
February 2026

Machine Learning



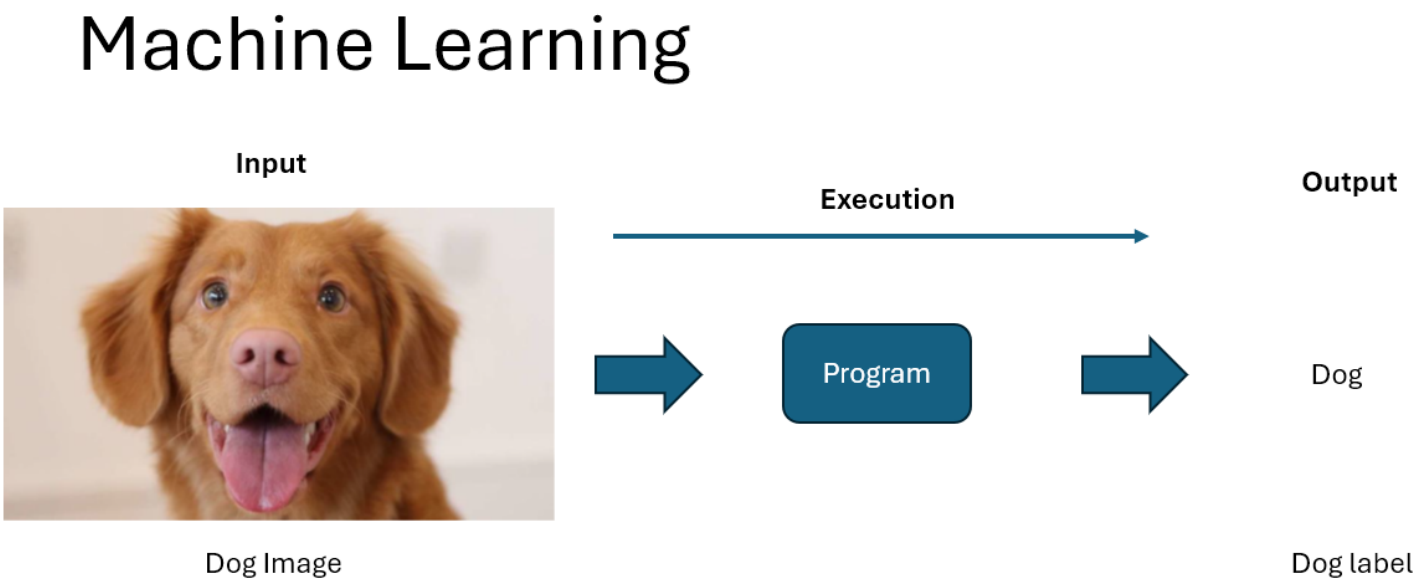
- A function is an operation/execution that maps inputs into outputs.
- The operation/execution can be defined by certain user specified rules (programming).
- What happens if the rules are not known or too complex to translate into code?

Machine Learning



- Instead of defining the **input/output** relationship by hand, define a program that acquires this relationship from data
- Can we learn the function?

Machine Learning



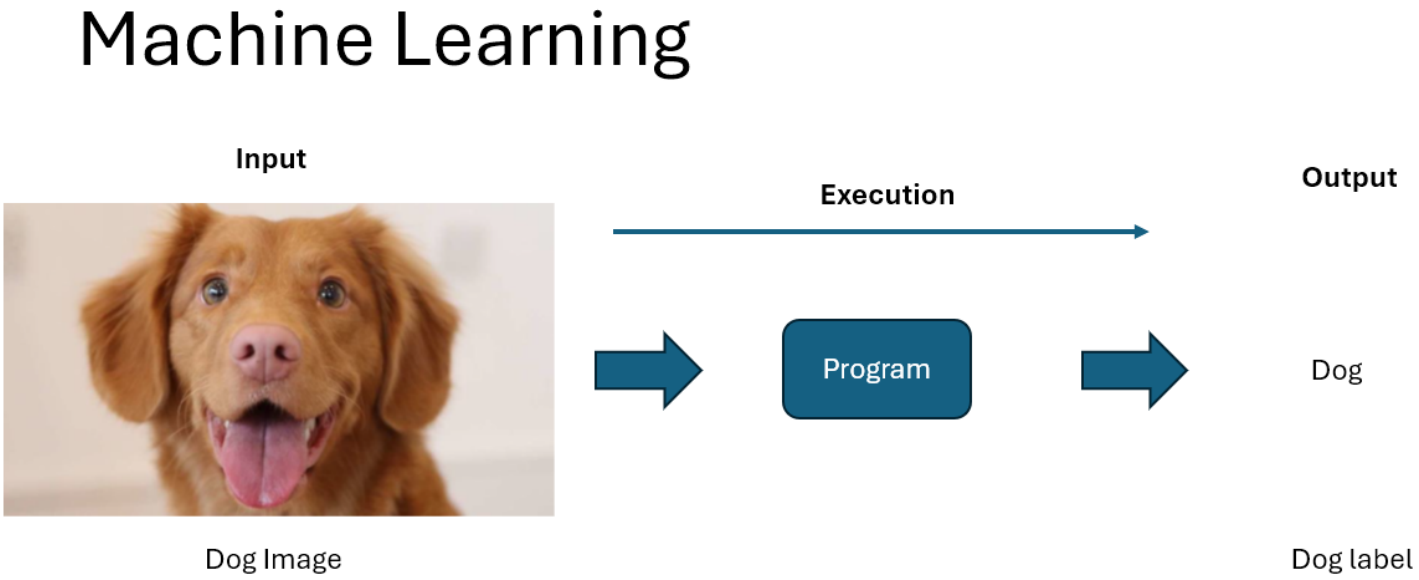
- We consider a function

$$f_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}$$

where:

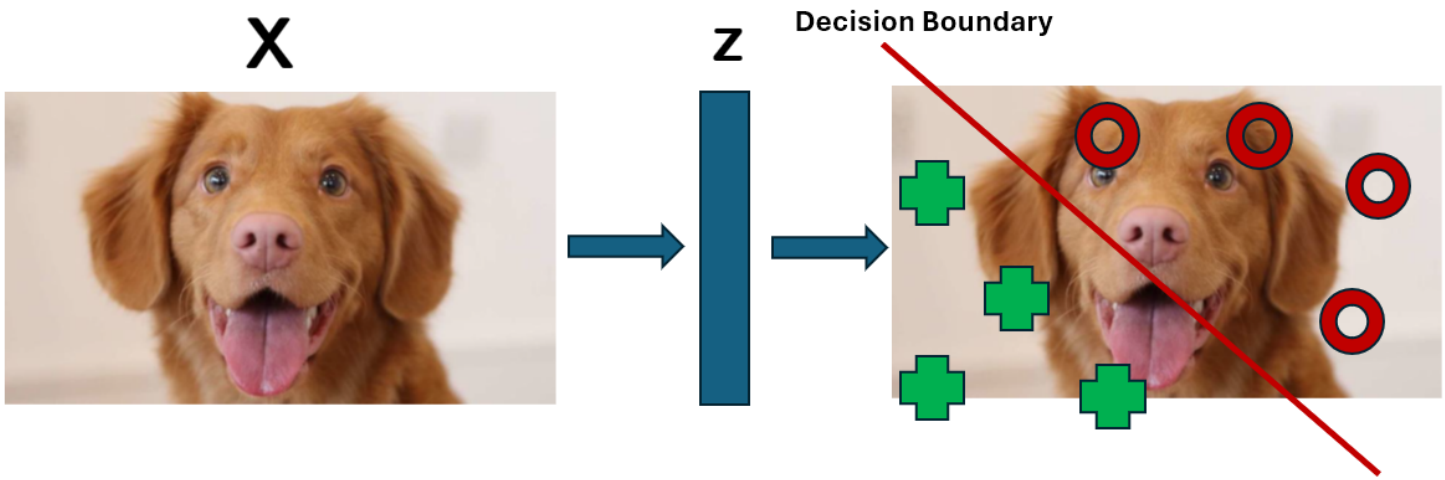
- $\mathcal{X}$ is the input space, $\mathcal{Y}$ is the output space, $\boldsymbol{\theta} \in \mathbb{R}^d$ are learnable parameters.

Machine Learning



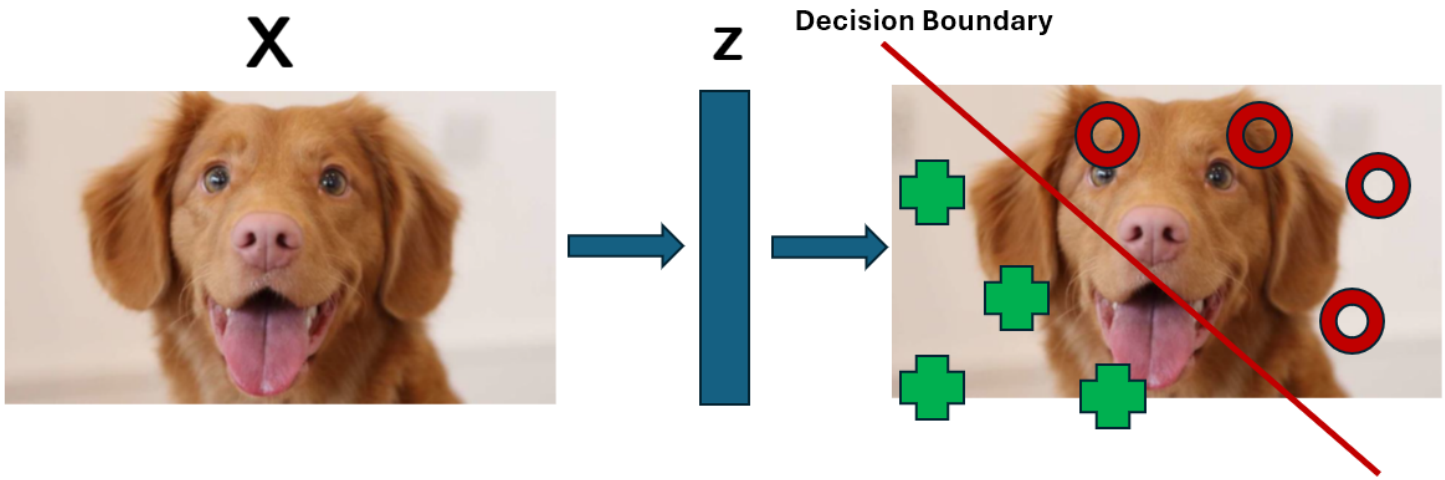
- Now assume that, given $\mathbf{X} \in \mathcal{X}$ we have a corresponding vector representation $\mathbf{z} \in \mathbb{R}^d$ (e.g., pixel values written as a column vector)

Machine Learning



- Rule: below the line (decision boundary) accept otherwise reject
- Defining the line: $z_1\theta_1 + z_2\theta_2 + z_3\theta_3 \leq 0$
- Objective: learn the best performing parameters $\boldsymbol{\theta} = [\theta_1, \theta_2, \theta_3]^T$.
- We use $f(\mathbf{z}; \boldsymbol{\theta}) = y$ or $f_{\boldsymbol{\theta}}(\mathbf{z}) = y$ to denote the parameterized function.

Machine Learning



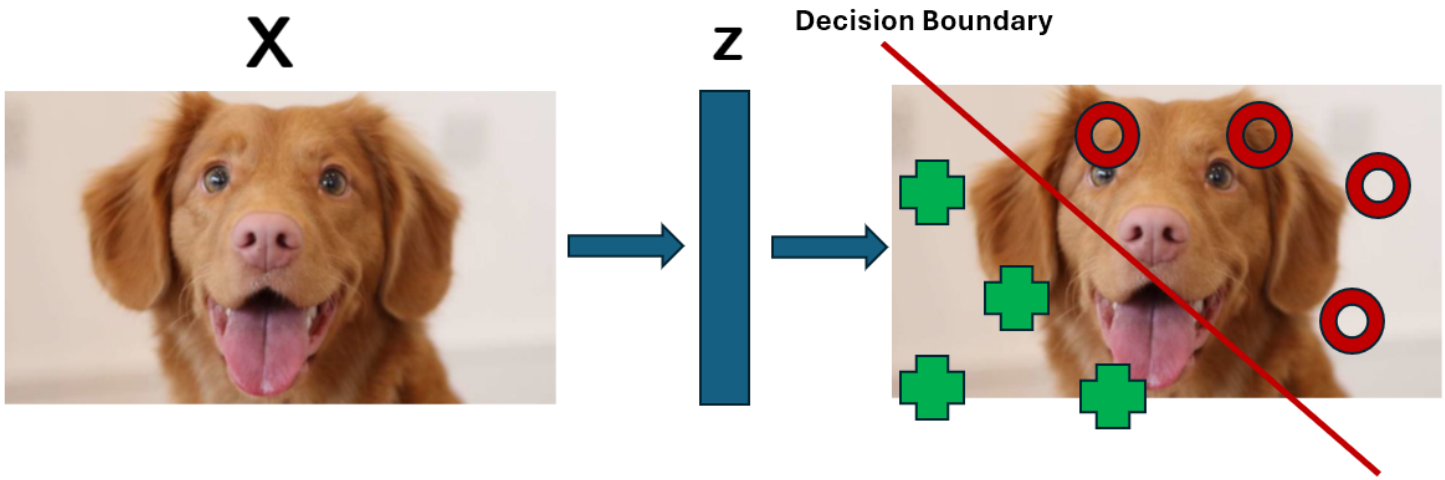
- Challenge: given a high dimensional data **X**, how can we transform it into a **representation z**?
- Formally, let

$$g : \mathcal{X} \to \mathcal{Z}$$

where:

- $\mathcal{X}$ is the input space, $\mathcal{Z}$ is the representation space and $g(\mathbf{X}) = z$

Machine Learning



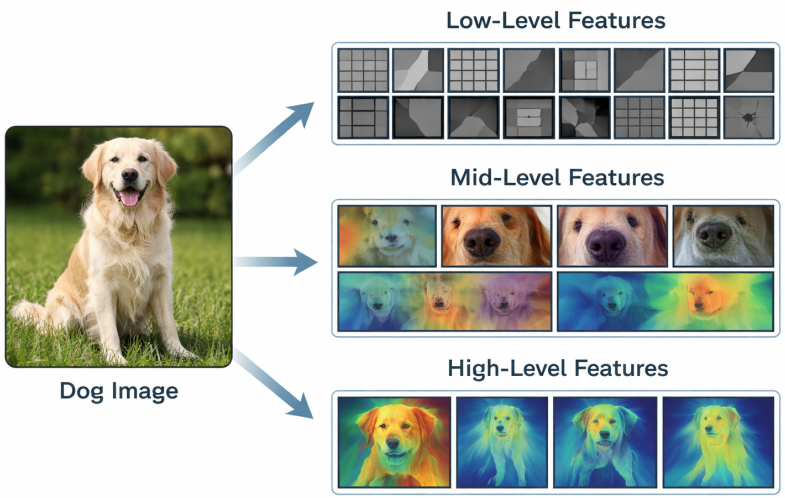
- Challenge: given a high dimensional data **X**, how can we transform it into a **representation z**?
- Formally, let

$$g : \mathcal{X} \to \mathcal{Z}$$

where:

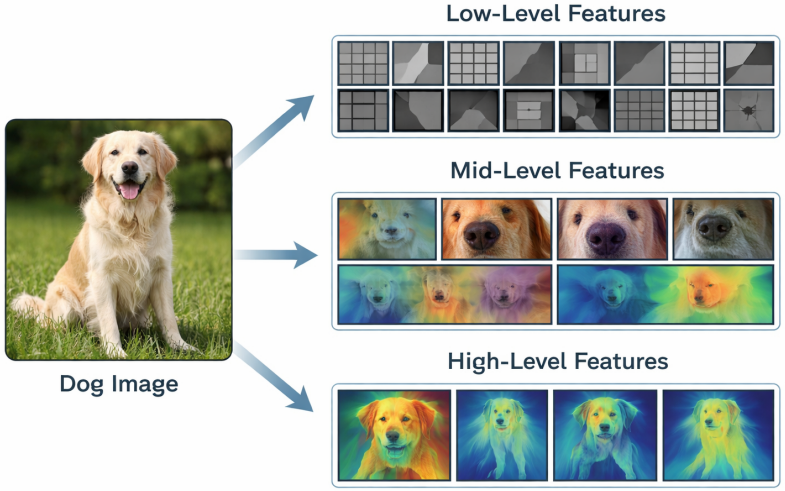
- $\mathcal{X}$ is the input space, $\mathcal{Z}$ is the representation space and $g(\mathbf{X}) = z$

Machine Learning



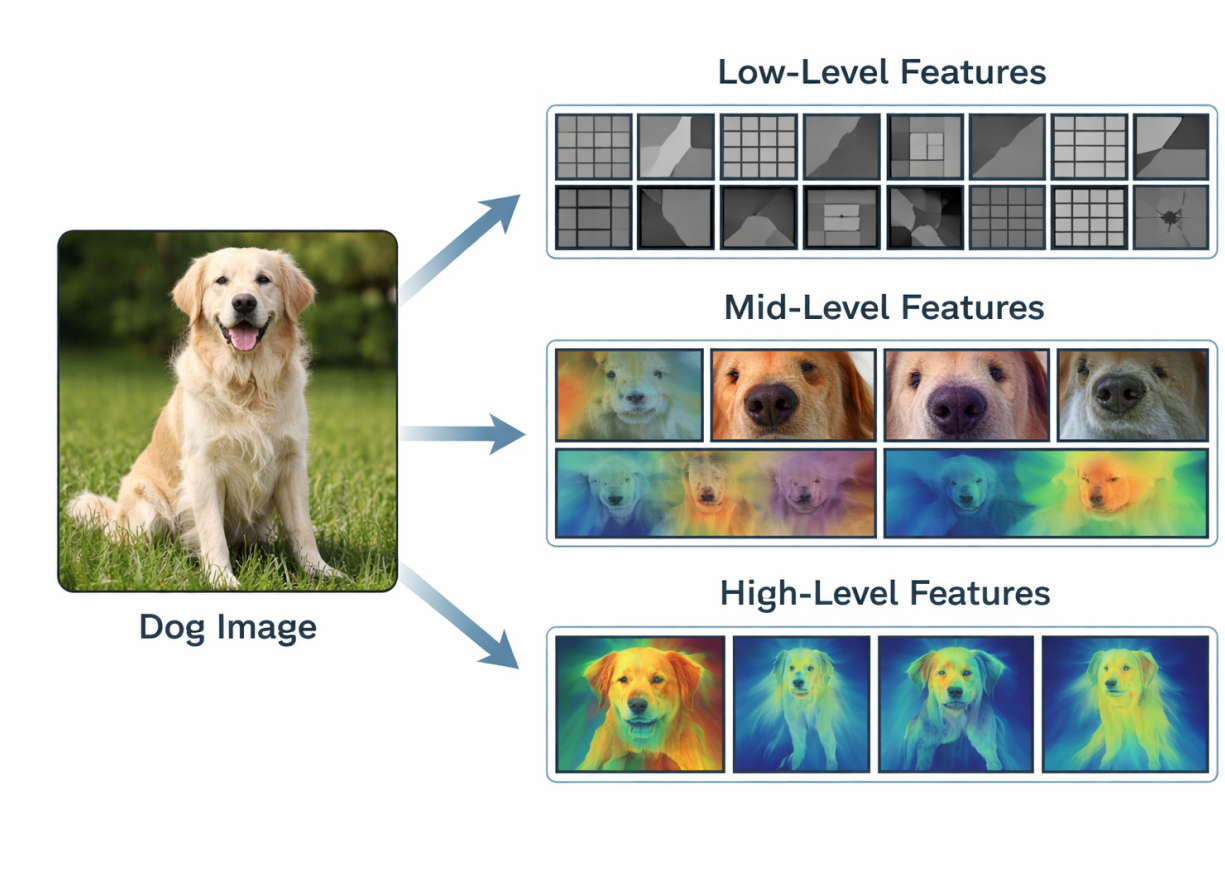
- Challenge: given a high dimensional data $\mathbf{X}$, how can we transform it into a **representation z**?
- Can we also make feature extraction part learnable, i.e., $g_\phi(\mathbf{X})$, where ϕ denotes the learnable parameters.
- Overall; we have a end-to-end learnable framework. $f_{\boldsymbol{\theta}}(g_\phi(\mathbf{X})) = y$

Deep Learning



- Machine learning with multiple layers of learned representations.
- The function that represents the transformation from input to internal representation to output is usually a deep neural network.
- The function has learnable parameters which are optimized with respect to the task of interest and the data set.

Deep Learning

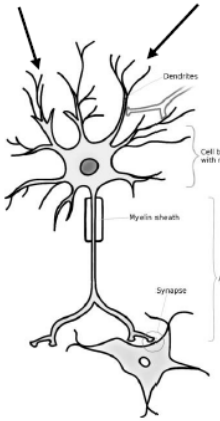


- Then, can we say that deep learning is an advance function approximation?

Brain vs Neural Networks

Early on, neural networks were proposed as a rudimentary model of neurons in the brain

dendrites receive signals from other neurons



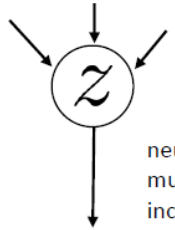
neuron "decides" whether to fire based on incoming signals

axon transmits signal to downstream neurons

artificial "neuron" sums up signals from upstream neurons (also referred to as "units")

$$z = \sum_i a_i$$

upstream activations

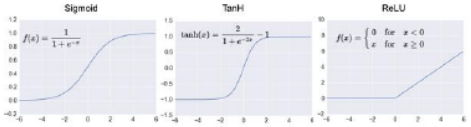


neuron "decides" how much to fire based on incoming signals

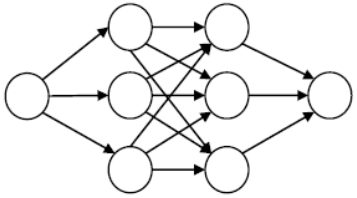
$$a = \sigma(z)$$

activations transmitted to downstream units

"activation function"



layer 1 layer 2



Is this a good model for real neurons?

- Crudely models *some* neuron function
- Missing many other important anatomical details
- Don't take it too seriously

- How effective such simplified model?

Universal Approximation Theorem [Cybenko 1989]

- Let I_n denote the n-dimensional unit cube, $[0,1]^n$.
- The space of continuous functions on I_n is denoted by $C(I_n)$ and we use $\|f\|$ to denote the supremum (or uniform) norm of an $f \in C(I_n)$. In general we use $\|\cdot\|$ to denote the maximum of a function on its domain.
- The space of finite, signed regular Borel measures on I_n is denoted by $\mathcal{M}(I_n)$.

Universal Approximation Theorem [Cybenko 1989]

Networks with one internal layer and an arbitrary **continuous sigmoidal function** can approximate continuous functions with arbitrary precision providing that no constraints are placed on the number of nodes or the size of the weights.

Theorem

Let σ be any continuous sigmoidal function. Then finite sums of the form

$$G(x) = \sum_{j=1}^{N} \alpha_j \, \sigma\Big(y_j^T x + \theta_j\Big)$$

are dense in $C(I_n)$. In other words, given any $f \in C(I_n)$ and $\varepsilon > 0$, there is a sum $G(x)$, of the above form, for which

$$|G(x) - f(x)| < \varepsilon \quad \textit{for all } x \in I_n.$$

Supervised Learning

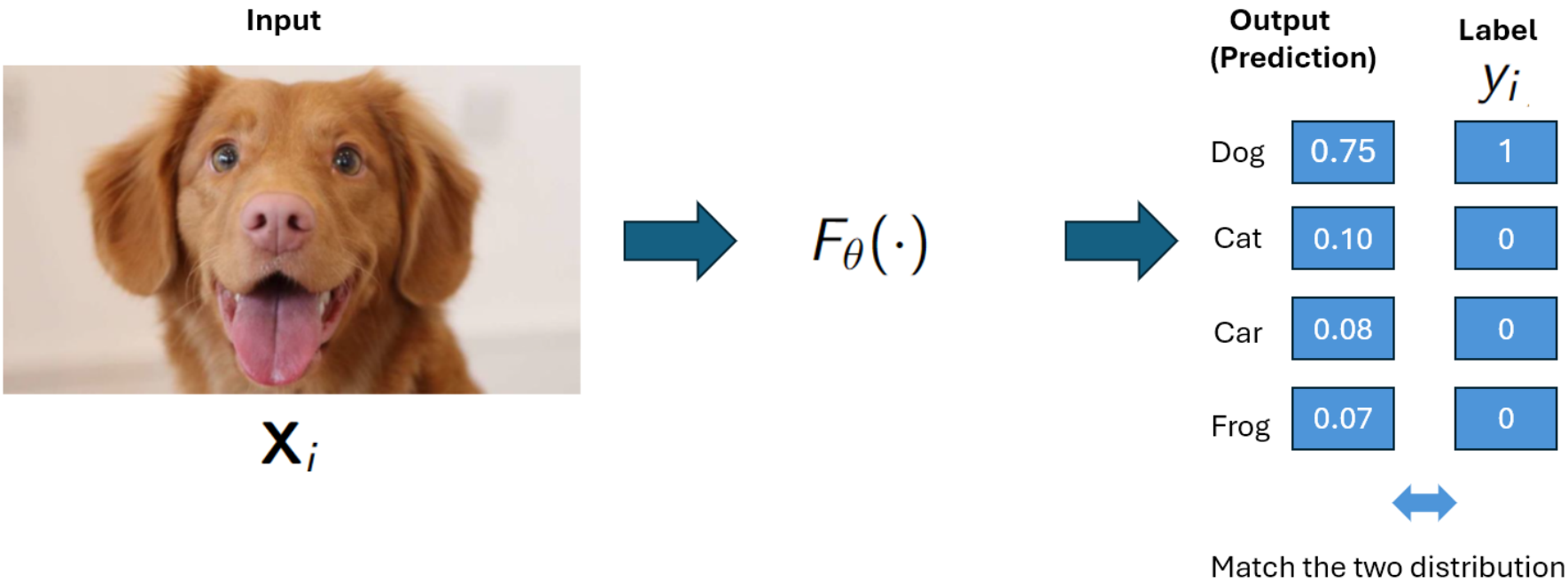
- Lets have a training dataset of length N, i.e., $\mathcal{D} = \{(\mathbf{X}_i, y_i)\}_{i=1}^{N}$ where $\mathbf{X}_i \in \mathcal{X}$ is the input, $y_i \in \mathcal{Y}$ is the corresponding label,
- The goal is to learn a function

$$F_\theta(\cdot) : \mathcal{X} \to \mathcal{Y} \tag{1}$$

  parameterized by θ, that approximates the true underlying mapping.
- The model is trained by minimizing the empirical risk:

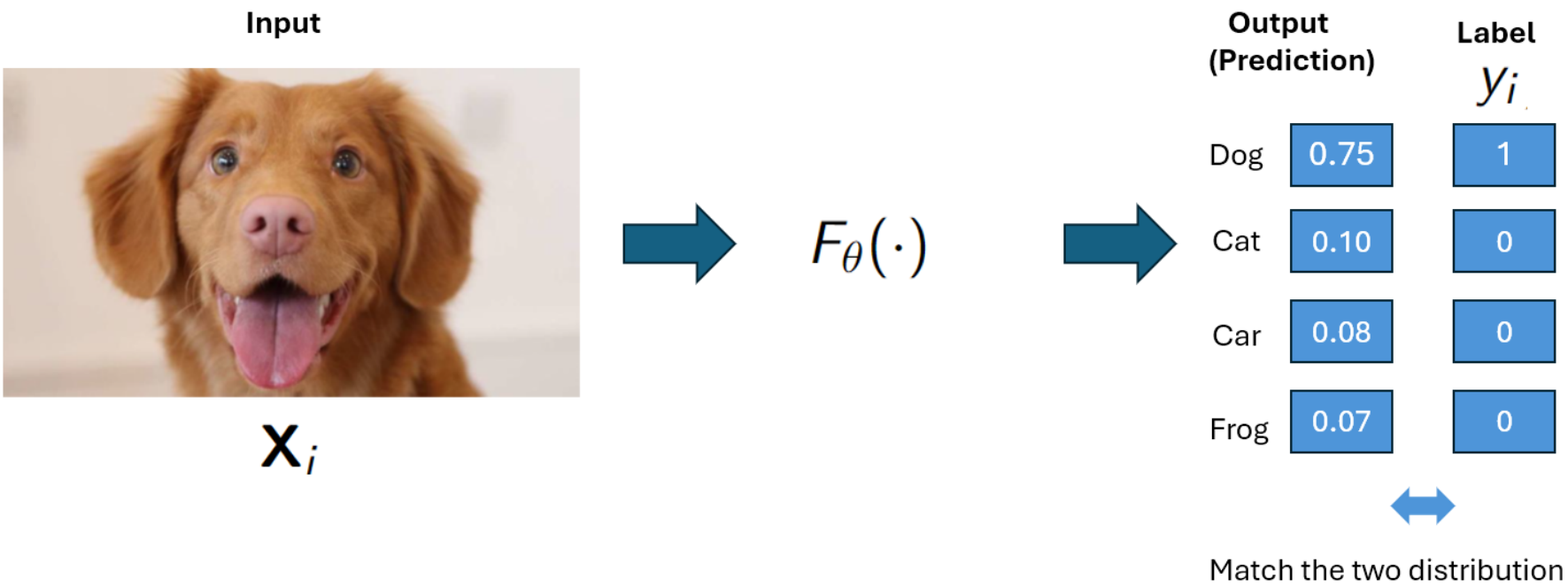
$$\min_\theta \ \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\underbrace{f_\theta(\mathbf{X}_i)}_{\tilde{y}_i}, y_i)$$

Supervised Learning (Image Classification)



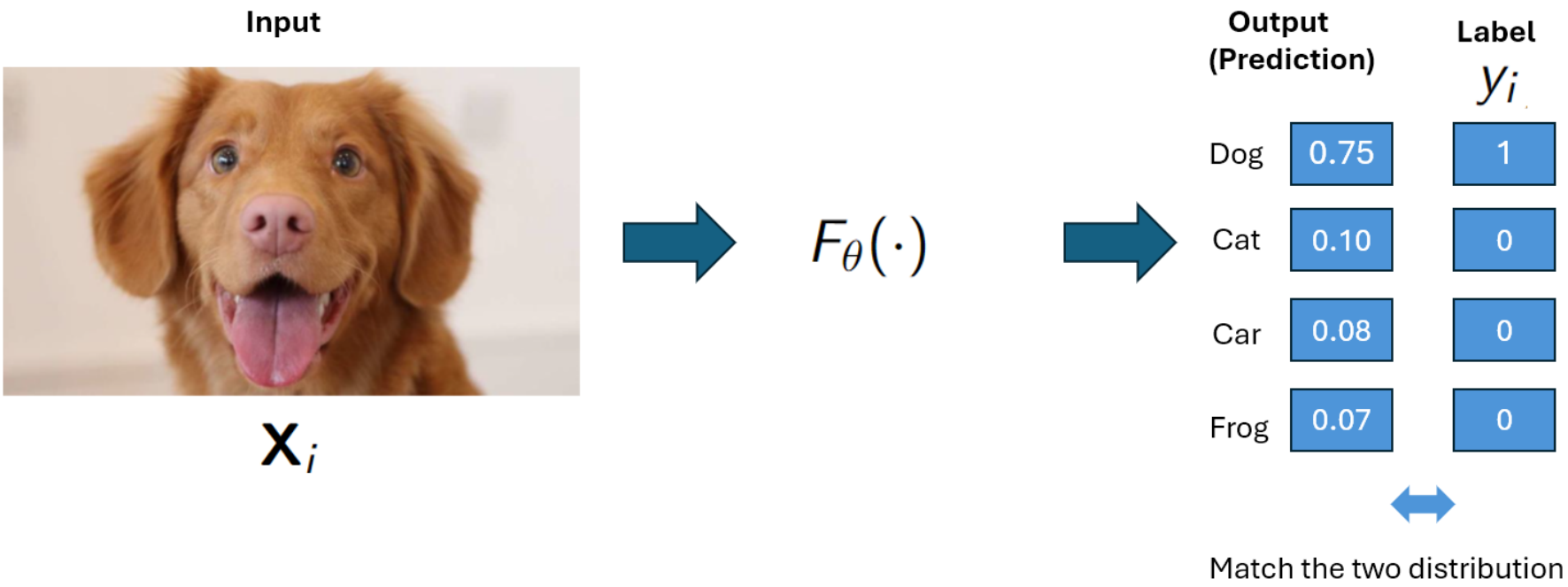
- Image classification is an example for supervised learning since the target label is available.

Supervised Learning (Image Classification)



- How can we design a classifier?
- Let $\mathbf{z} \in \mathbb{R}^d$ be the penultimate (final layer) latent representation for features.
- Then, having parametrized d dimensional vectors for each class, e.g., $\boldsymbol{\theta}_{cat}, \boldsymbol{\theta}_{dog}, \boldsymbol{\theta}_{car}, \boldsymbol{\theta}_{frog}$, one can compute $\mathbf{z}^T \boldsymbol{\theta}_{class}$ to understand how features are aligned with a particular class.

Supervised Learning (Image Classification)



- Lets us have all the logit values; $\tilde{z}_{cat} = \mathbf{z}^T \boldsymbol{\theta}_{cat}$,$\tilde{z}_{dog} = \mathbf{z}^T \boldsymbol{\theta}_{dog}$ $\tilde{z}_{car} = \mathbf{z}^T \boldsymbol{\theta}_{car}$ $\tilde{z}_{frog} = \mathbf{z}^T \boldsymbol{\theta}_{frog}$
- Now, how can we map the logits to the corresponding probabilities?

Softmax classifier:

- Given a vector of logits $z \in \mathbb{R}^C$, the softmax function is defined as:

$$\mathrm{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, \ldots, C.$$

$$p_i \geq 0, \qquad \sum_{i=1}^{K} p_i = 1$$

- That is, $p_{dog} = \frac{e^{\tilde{z}_{dog}}}{e^{\tilde{z}_{dog}} + e^{\tilde{z}_{cat}} + e^{\tilde{z}_{car}} + e^{\tilde{z}_{frog}}}$,

Softmax classifier:

- Softmax maps $\mathbb{R}^d$ to the probability simplex:

$$\mathrm{Softmax} : \mathbb{R}^C \to \Delta^{C-1}.$$

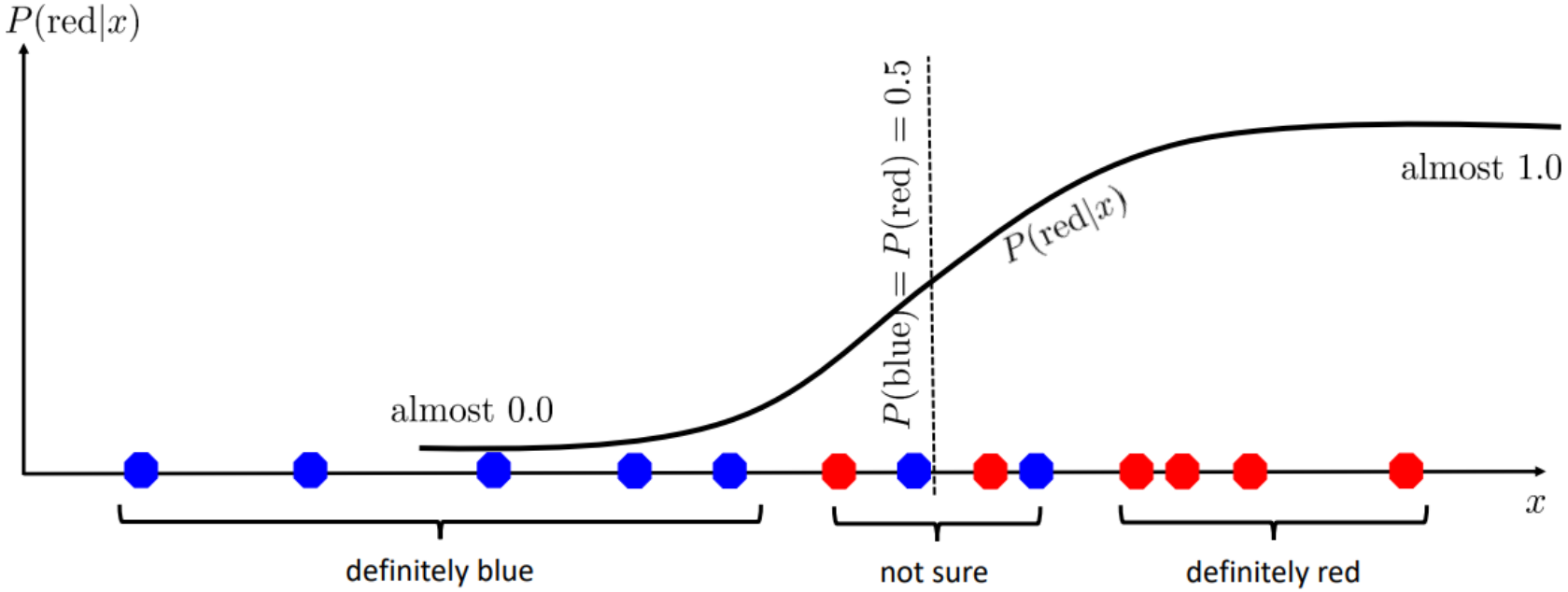
- Temperature Softmax:

$$\mathrm{Softmax}(z_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

  - $T < 1$: sharper distribution
  - $T > 1$: smoother distribution

Softmax Illustration

An illustration: 1D case

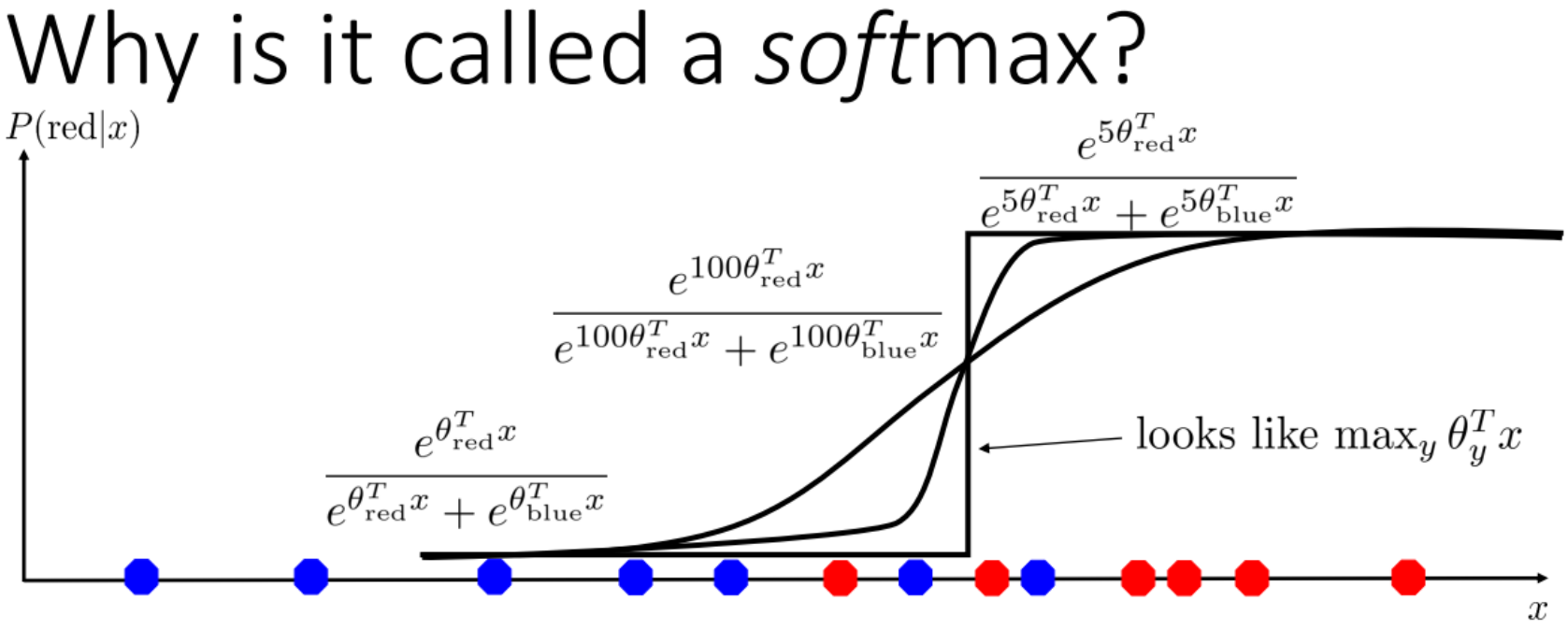


$$P(\text{red}|x) = \frac{e^{\theta_{\text{red}}^T x}}{e^{\theta_{\text{red}}^T x} + e^{\theta_{\text{blue}}^T x}}$$

probability increases exponentially as we move away from boundary

normalizer

Softmax Illustration



$$P(\text{red}|x) = \frac{e^{\theta_{\text{red}}^T x}}{e^{\theta_{\text{red}}^T x} + e^{\theta_{\text{blue}}^T x}}$$

Cross-Entropy Loss (Inf. Theoretic perspective)

- How can we match the true and predicted distribution?
- Entropy: for a discrete random variable X with distribution $p(x)$, the entropy is defined as

$$H(X) = -\sum_{x} p(x) \log p(x) \tag{2}$$

- Entropy measures the uncertainty of a random variable
- The Kullback-Leibler (KL) divergence: measures how different a probability distribution Q is from a true distribution P.

$$D_{KL}(P \| Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)} \tag{3}$$

Cross-Entropy Loss (Inf. Theoretic perspective)

- Which can be also written as:

$$D_{KL}(P\|Q) = H(P,Q) - H(P) \tag{4}$$

where

$$H(P) = -\sum_x P(x)\log P(x) \tag{5}$$

is the entropy of P, and

$$H(P,Q) = -\sum_x P(x)\log Q(x) \tag{6}$$

is the cross-entropy between P and Q.

Cross-Entropy Loss (Inf. Theoretic perspective

- Lets fix a single data $\mathbf{X}_i$, let $p(y|\mathbf{X}_i)$ be true label distribution and $p_{\boldsymbol{\theta}}(y|\mathbf{X}_i)$ be the model based label distribution (predicted). Then, the KL divergence:

$$\underbrace{\sum_y p(y|\mathbf{X}_i) \log p(y|\mathbf{X}_i)}_{\text{Do not depend } \boldsymbol{\theta}} - \sum_y p(y|\mathbf{X}_i) \log p_{\boldsymbol{\theta}}(y|\mathbf{X}_i) \tag{7}$$

- Since the first term does not depend on $\boldsymbol{\theta}$, our target is the second term which is the cross entropy.
- Now since $p(y|\mathbf{X}_i) = 0$ for $y \neq y_i$, we want to maximize $\log p_{\boldsymbol{\theta}}(y_i|\mathbf{X}_i)$

Maximum Likelihood (ML) estimation

- Assume data samples are coming from an unknown joint distribution $(x, y) \sim p(x, y)$
- Also assume we have randomly drawn n samples $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$
- Under IID assumption, we have $p(\mathcal{D}) = p\big((x_1, y_1), \ldots, (x_n, y_n)\big) = \prod_{i=1}^{n} p(x_i, y_i) = \prod_i p(x_i)\, p(y_i \mid x_i)$
- We aim to learn a parametric model: $p_\theta(y \mid x)$
- ML estimate: maximize the occurence probability of sampled dataset

$$\log p(\mathcal{D}) = \sum_i \log p(x_i) + \log p_\theta(y_i \mid x_i) = \sum_i \log p_\theta(y_i \mid x_i) + \mathit{const} \tag{8}$$

Maximum Likelihood (ML) estimation

- Maximum likelihood estimation (MLE):

$$\theta^\star \leftarrow \arg\max_\theta \sum_i \log p_\theta(y_i \mid x_i) \tag{9}$$

- Negative log-likelihood (NLL)

$$\theta^\star \leftarrow \arg\min_\theta - \sum_i \log p_\theta(y_i \mid x_i) \tag{10}$$

Do we use NLL loss only for classification ?

- Assume the regression model

$$y = f_\theta(x) + \varepsilon$$

where the noise is Gaussian:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2).$$

- Then the conditional distribution of y given x is

$$p(y \mid x; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f_\theta(x))^2}{2\sigma^2}\right). \tag{11}$$

- Given a dataset $\{(x_i, y_i)\}_{i=1}^n$ with i.i.d. samples, the likelihood is

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} p(y_i \mid x_i; \theta). \tag{12}$$

- Then, taking the log-likelihood:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^{n} \left[-\frac{(y_i - f_\theta(x_i))^2}{2\sigma^2} - \frac{1}{2}\log(2\pi\sigma^2)\right]. \tag{13}$$

Do we use NLL loss only for classification ?

- Assume the regression model

$$y = f_\theta(x) + \varepsilon$$

where the noise is Gaussian:

$$\epsilon \sim \mathcal{N}(0, \sigma^2) \text{ and } y \sim \mathcal{N}(f_\theta(x), \sigma^2) \tag{14}$$

- Then the conditional distribution of y given x is

$$p(y \mid x; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f_\theta(x))^2}{2\sigma^2}\right). \tag{15}$$

- Given a dataset $\{(x_i, y_i)\}_{i=1}^n$ with i.i.d. samples, the likelihood is

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} p(y_i \mid x_i; \boldsymbol{\theta}). \tag{16}$$

- Then, taking the log-likelihood:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^{n} \left[-\frac{(y_i - f_{\boldsymbol{\theta}(x_i)})^2}{2\sigma^2} - \frac{1}{2}\log(2\pi\sigma^2)\right]. \tag{17}$$

Do we use NLL loss only for classification ?

- Negative log-likelihood:

$$NLL(\boldsymbol{\theta}) = \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left[(y_i - f_{\boldsymbol{\theta}(x_i)})^2 \right] + \text{const} \tag{18}$$

- Then minimizing NLL is equivalent to minimizing MSE:

$$MSE(\boldsymbol{\theta}) = \sum_{i=1}^{n} \left[(y_i - f_{\boldsymbol{\theta}(x_i)})^2 \right] \tag{19}$$

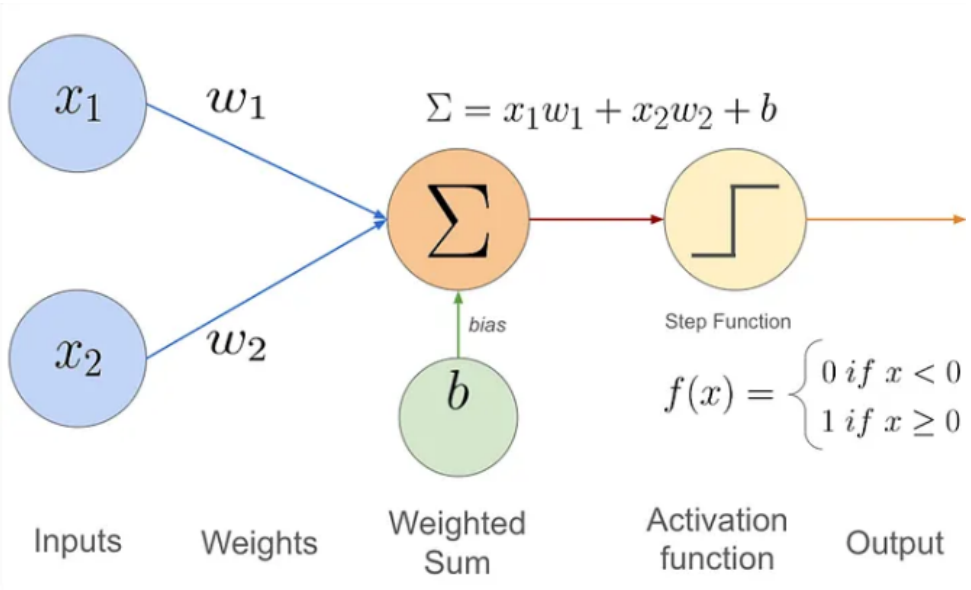
- When the task is linear regression

$$MSE = (\boldsymbol{\theta}) \sum_{i=1}^{n} (y_i - \boldsymbol{\theta}^T \mathbf{x}_i + b)^2 \tag{20}$$

Short Summary So Far

- We have 3 main aspects in deep learning
- Define a model with learnable parameters
- Define the task-specific loss
- Update the parameters towards minimizing the loss.
- Now, we revisit some of the earlier models.

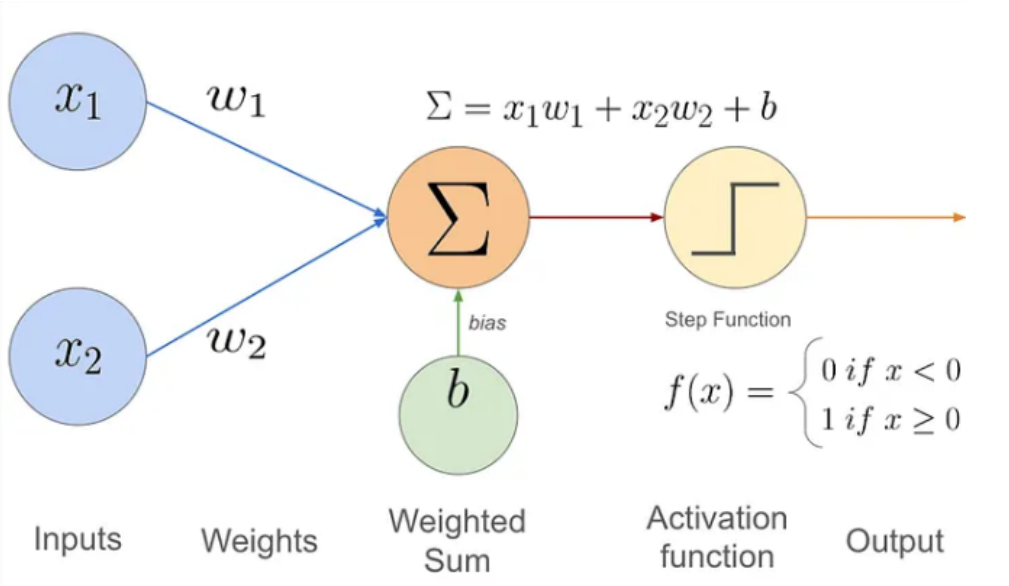
Perceptron



Two-class Linear classification:

$$\hat{y}_i = f(\mathbf{x}_i; \mathbf{w}) \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x}_i > 0 \\ 0 & \text{otherwise} \end{cases} \tag{21}$$

Perceptron



Perceptron Learning Algorithm: for a given $(\mathbf{x}_i, y_i)$, update model parameters $\mathbf{w}$:

$$\mathbf{w}^{\text{next}} = \mathbf{w}^{\text{curr}} + \eta(y_i - \hat{y}_i)\mathbf{x}_i \tag{22}$$

where:

- $\eta > 0$ is the learning rate,
- $y_i \in \{0, 1\}$ is the true label,
- $\hat{y}_i$ is the predicted label.

Perceptron (learning)

- Recall:

$$\mathbf{w}^{\text{next}} = \mathbf{w}^{\text{curr}} + \eta(y_i - \hat{y}_i)\mathbf{x}_i \tag{23}$$

- If $y_i = \hat{y}_i$, then $\mathbf{x}_i$ is correctly classified and since $y_i - \hat{y}_i = 0$, $\mathbf{w}$ won't change.
- For false classified $\mathbf{x}_i$:
  - if $y_i = 1$, then $\eta(y_i - \hat{y}_i) > 0$
  - if $y_i = 0$, then $\eta(y_i - \hat{y}_i) < 0$
- Now multiply both side of learning equation with $\mathbf{x}_i^T$

$$\mathbf{x}_i^T\mathbf{w}^{\text{next}} = \mathbf{x}_i^T\mathbf{w}^{\text{curr}} + \eta(y_i - \hat{y}_i)\mathbf{x}_i^T\mathbf{x}_i$$

- $\mathbf{x}_i^T\mathbf{x}_i > 0$,
  - for $y_i = 1$ we observe

$$\mathbf{x}_i^T\mathbf{w}^{\text{next}} > \mathbf{x}_i^T\mathbf{w}^{\text{curr}}$$

  - for $y_i = 0$ we observe

$$\mathbf{x}_i^T\mathbf{w}^{\text{next}} < \mathbf{x}_i^T\mathbf{w}^{\text{curr}}.$$

Limitations of linear classifiers

- A linear classifier has the form

$$f(x) = \mathbf{w}^\top \mathbf{x} + b, \tag{24}$$

  and predicts

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0, \\ 0 & \text{otherwise.} \end{cases}$$

- Define the positive decision region:

$$C = \{\mathbf{x} \in \mathbb{R}^d \mid \mathbf{w}^\top \mathbf{x} + b \geq 0\}.$$

- A set $C \subset \mathbb{R}^d$ is convex if

$$\forall \mathbf{x}_1, \mathbf{x}_2 \in C, \quad \forall \lambda \in [0, 1],$$

$$\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in C.$$

Limitations of linear classifiers

- Lets show C is Convex: assume $\mathbf{x}_1, \mathbf{x}_2 \in C$, Then $\mathbf{w}^\top \mathbf{x}_1 + b \geq 0, \quad \mathbf{w}^\top \mathbf{x}_2 + b \geq 0.$
- Now take any $\lambda \in [0, 1]$ and define

$$\mathbf{x}_\lambda = \lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2.$$

- Evaluate the classifier:

$$\mathbf{w}^\top \mathbf{x}_\lambda + b = \mathbf{w}^\top (\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2) + b.$$

- Using linearity:

$$= \lambda \mathbf{w}^\top \mathbf{x}_1 + (1 - \lambda)\mathbf{w}^\top \mathbf{x}_2 + b.$$

- Rewriting:

$$= \lambda(\mathbf{w}^\top \mathbf{x}_1 + b) + (1 - \lambda)(\mathbf{w}^\top \mathbf{x}_2 + b).$$

Limitations of linear classifiers

- Since both $(\mathbf{w}^\top \mathbf{x}_1 + b)$ and $(\mathbf{w}^\top \mathbf{x}_2 + b)$ are nonnegative and $\lambda \in [0, 1]$, we obtain

$$\mathbf{w}^\top \mathbf{x}_\lambda + b \geq 0.$$

- Therefore, $\mathbf{x}_\lambda \in CandC$ is convex.
- Every linear classifier defines a half-space, and every half-space is convex.
- Suppose a classification problem has a positive class region C that is non-convex.
- Then there exist $\mathbf{x}_1, \mathbf{x}_2 \in C$ and some $\lambda \in (0, 1)$ such that

$$\mathbf{x}_\lambda = \lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \notin C. \tag{25}$$

- But we proved that any region defined by a linear classifier must be convex. This is a contradiction.

Example Case: XOR

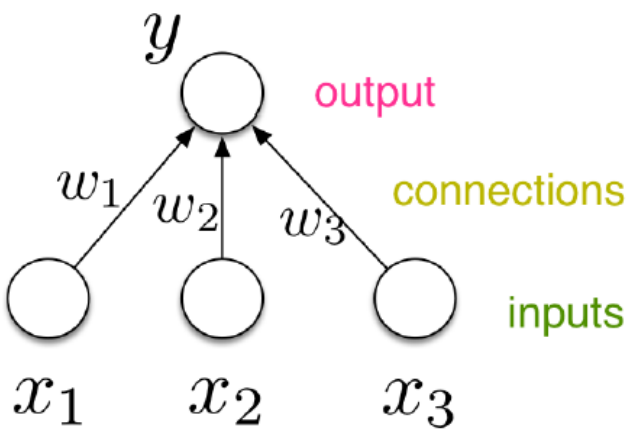
- Label the XOR dataset as:

$$\text{Positive class: } (0,1),(1,0),$$

$$\text{Negative class: } (0,0),(1,1).$$

- Assume there exists a separating hyperplane $f(x) = w_1 x_1 + x_2 w_2 + b$.
- From the positive points:
  - (1) $w_2 + b > 0 \quad (\text{for } (0,1))$,
  - (2) $w_1 + b > 0 \quad (\text{for } (1,0))$.
- From the negative points:
  - (3) $b < 0 \quad (\text{for } (0,0))$,
  - (4) $w_1 + w_2 + b < 0 \quad (\text{for } (1,1))$.
- (1), (2) and (4) implies $b > 0$ while (3) implies $b < 0$, hence, contradiction !

Beyond Perceptron



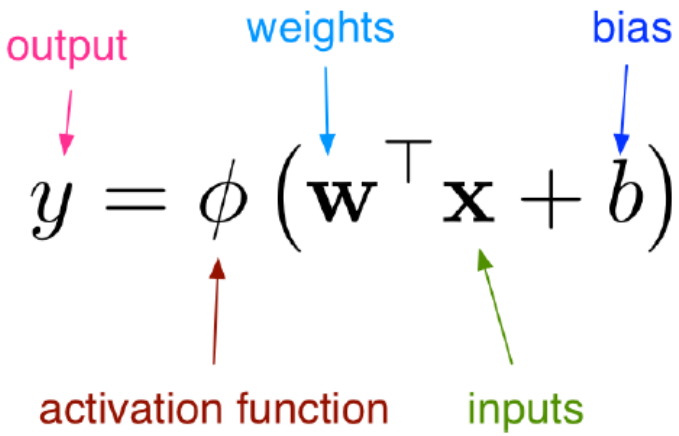


Figure 1: Illustration of a general neuron-like unit

- Using sigmoidal activations, we can approximate wider range of functions (recall universal approximation thm.)
- Connect a large number of units together into a directed acyclic graph for better approximation (deeper neural architectures).

Multi-layer perceptron (MLP)

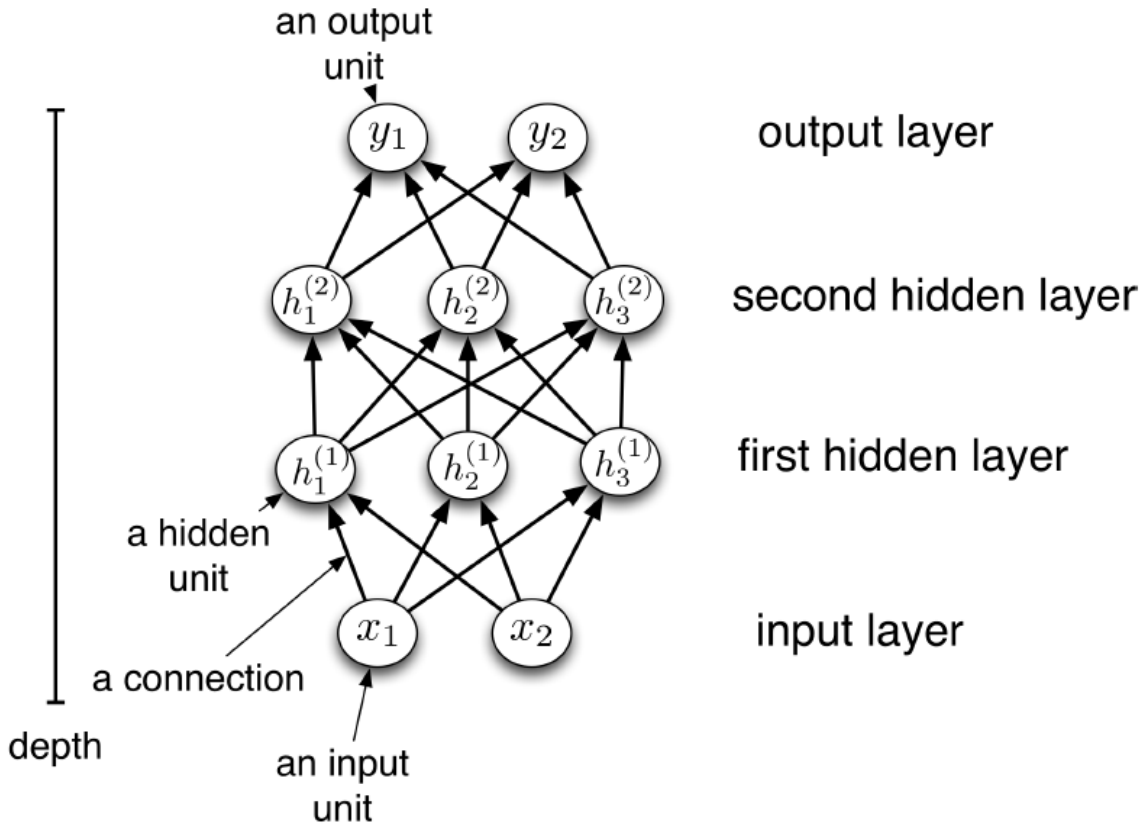


Figure 2: Illustration of a two layer MLP architecture (Roger Grosse and Jimmy Ba CSC 421)

- Now, we have a new term layer, which identifies the stage of the information processing.

Multi-layer perceptron (MLP)

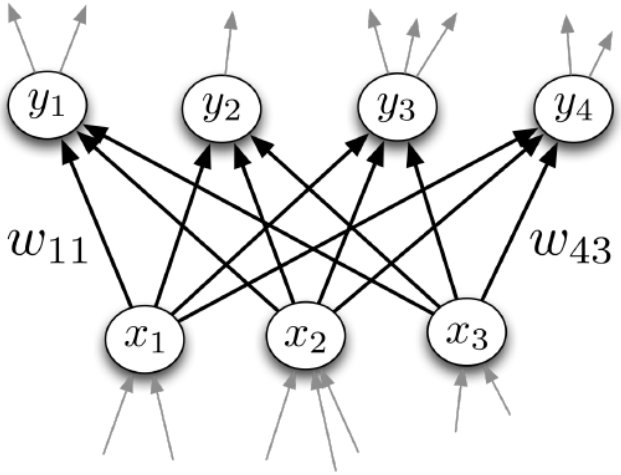


Figure 3: Connection of layers (Roger Grosse and Jimmy Ba CSC 421)

- Each layer connects N input units to M output units (neurons).
- Ideally, we use $N \times M$ weights to connect N units in the $l-1$th layer to M units in lth layer, and use $w_{i,j}$ to denote the weight connecting ith unit in lth layer to jth unit in $l-1$th layer.

Multi-layer perceptron (MLP)

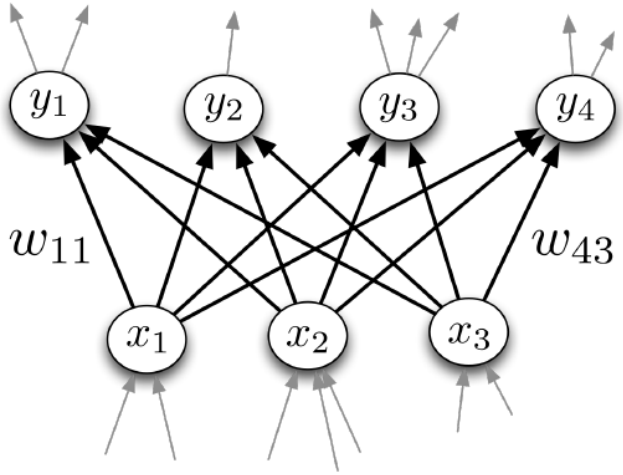


Figure 4: Connection of layers (Roger Grosse and Jimmy Ba CSC 421)

- The input processing between the lth and $l-1$th layer given as

$$\mathbf{x}_l = \mathbf{y}_{l-1} = f_{l-1}(\mathbf{x}_{l-1}) = \phi(\mathbf{W}\mathbf{x}_{l-1} + \mathbf{b}_{l-1}) \tag{26}$$

where $\mathbf{x}_l \in \mathbb{R}^M, \mathbf{x}_{l-1} \in \mathbb{R}^N$ and $\mathbf{W} \in \mathbb{R}^{M \times N}$

Activation functions Sigmoid

- Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{27}$$

- It maps real numbers to $(0, 1)$.
- The output can be interpreted as probability.
- Can be used as classifier with binary cross-entropy loss.
- Derivative: $\frac{d}{dx}\sigma(x) = \sigma(x)(1 - \sigma(x))$
- Computationally efficient, but suffers vanishing gradient problem

Activation functions: RelU

- ReLU:
$$\sigma(x) = \max(0, x) \tag{28}$$
- Derivative $\frac{d}{dx}\sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$
- For $x > 0$, the gradient is 1, so gradients do not vanish as in sigmoid
- Requires only a threshold operation
- **Sparse activation:** Since negative inputs become zero, many neurons remain inactive (better generalization).
- **Dying ReLU problem:** If a neuron consistently receives negative input, the gradient becomes zero and may become inactive.
- **Not zero-centered:** Output is always non-negative, which may slow optimization.
- **Unbounded:** Large positive inputs produce very large activations, which may lead to insatiability.

Activation functions: Leaky RelU

- Leaky ReLU: $\sigma(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}$ where α is a small positive constant (typically $\alpha = 0.01$)
- Prevents the **dying ReLU** problem.
- Allows small gradient flow for negative inputs.
- Improves training stability in deep networks.
- **Smoothness:** The function is not differentiable at $z = 0$.

Activation functions

- ELU:

$$\sigma(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases} \tag{29}$$

  where $\alpha > 0$ is a hyperparameter (commonly $\alpha = 1$)

- Derivative:

$$\frac{d}{dx}\sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha e^x & \text{if } x \leq 0 \end{cases} \tag{30}$$

- **Non-zero gradient for negative inputs:** Prevents dying neuron problem.
- **Negative outputs:** Activations can be negative, making them closer to zero-centered, speeds up learning.
- **Smoother than ReLU:** The function is continuous and differentiable everywhere.
- Computationally more costly.

Activation functions

- GeLU:

$$f(x) = x\Phi(x) \tag{31}$$

where:

$$\Phi(x) = \int_{-\infty}^{x} \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt \tag{32}$$

Related and suggested reading materials

- Berkeley CS 182 (Sergey Levine)
  https://cs182sp21.github.io/
- METU CEng 501 (Emre Akbas)
  https://user.ceng.metu.edu.tr/~emre/Fall2025-DeepLearning.html
- Uni. Toronto
  https://www.cs.toronto.edu/~rgrosse/courses/csc421_2019/